12.1 协议设计哲学

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~200,000 tokens，输出 ~15,000 tokens（本章合计）

第 3 章里我们看到了 Gateway 如何启动 WebSocket 和 HTTP 服务器。但"有一条 WebSocket 连接"和"客户端与服务器能有效通信"之间还有巨大的鸿沟。本节分析 OpenClaw 如何在 WebSocket 之上设计了一套完整的应用层协议，以及这些设计决策背后的权衡。

> **术语说明**：本章中频繁出现的"客户端"（Client）一词，在 Gateway 协议语境下特指**通过 WebSocket 连接到 Gateway 的程序**——包括 macOS 菜单栏应用、CLI 终端、Web 控制台、iOS/Android 节点设备等。它们是 OpenClaw **操作者**（部署和管理 OpenClaw 的人）使用的监控与控制工具。请注意将其与"消息通道"（Channels）区分开来：通道是面向终端用户的消息入口（如 WhatsApp、Telegram），它们嵌入在 Gateway 进程内部，与 Gateway 之间是进程内函数调用而非 WebSocket 通信。详见第 3.1.1 节的连接管理部分。

12.1.1 为什么选择 WebSocket 而非 REST

先回顾一个基础问题：为什么要用 WebSocket?

REST 的局限性

REST（Representational State Transfer）是 Web 应用中最常见的 API 设计风格。在 REST 架构中，客户端发起 HTTP 请求，服务器返回响应，然后连接关闭。这种"请求-响应"模型简单、无状态、易于缓存和扩展，非常适合 CRUD（增删改查）操作。

但对于 AI 助手这类应用，REST 有几个根本性的局限：

1. 无法实现服务器主动推送

当 Agent 正在运行时，会产生一系列中间事件——工具调用开始了、模型返回了一个文本块、工具执行完毕了。这些事件需要**实时**推送给客户端。在 REST 模型中，客户端只能通过轮询（Polling）来获取这些事件，效率低下且延迟高。

> **衍生解释**：轮询（Polling）是客户端定期向服务器发送请求来检查是否有新数据的方式。虽然 HTTP 长轮询（Long Polling）和 Server-Sent Events（SSE）可以部分解决这个问题，但前者仍然

> 需要频繁建立连接，后者只支持服务器到客户端的单向通信。

2. 请求-响应模型不匹配 AI 交互模式

一次 AI 对话可能持续数分钟（Agent 可能要执行多个工具调用、等待审批等）。在 REST 模型中，这意味着要么维持一个超长的 HTTP 连接等待响应，要么拆分成"启动任务"和"查询结果"两个接口。

3. 多路复用需求

OpenClaw 需要在同一个连接上同时进行：发起 Agent 对话、查询会话列表、管理配置、接收状态事件……如果用 REST，每个操作都需要独立的 HTTP 请求。

WebSocket 的优势

WebSocket 是 HTML5 标准定义的全双工通信协议。它在一次 HTTP 升级握手之后，建立持久的 TCP 连接，双方可以随时主动发送消息。

```
┌──────────┐                    ┌──────────┐
│  Client  │ ── HTTP Upgrade ──▶│  Server  │
│          │◀── 101 Switching ──│          │
│          │                    │          │
│          │◀──── 双向消息 ────▶│          │
│          │                    │          │
└──────────┘                    └──────────┘
```

对 OpenClaw 来说，WebSocket 的核心优势：

特性	REST	WebSocket
连接模式	每次请求新建连接	持久长连接
通信方向	客户端→服务器	双向
服务器推送	需要轮询或 SSE	原生支持
多路复用	不支持（需多次请求）	单连接多路复用
连接开销	每次都有 HTTP 头	一次握手后极低

OpenClaw 的选择：WebSocket + HTTP 互补

OpenClaw 并没有完全放弃 HTTP。如第 3.4 节所述，Gateway 同时提供 HTTP 端点用于：

- **OpenAI 兼容 API**（`/v1/chat/completions`）：为了与现有工具链兼容
- **Webhook 回调**（`/hook/`）：某些通道（如 Slack）需要 HTTP 回调
- **健康检查**（`/health`）：标准运维需求

这是一种务实的设计——用 WebSocket 处理**有状态的、实时的**交互，用 HTTP 处理**无状态的、兼容性的**需求。

12.1.2 请求-响应 + 服务器推送事件的混合模式

WebSocket 协议本身只定义了"发送消息"的能力，并没有内置"请求-响应"的概念。OpenClaw 在 WebSocket 之上设计了一套应用层帧（Frame）协议来解决这个问题。

三种帧类型

OpenClaw 协议定义了三种顶层帧类型，通过 `type` 字段区分：

```typescript
// src/gateway/protocol/schema/frames.ts

// 请求帧：客户端 → 服务器
export const RequestFrameSchema = Type.Object({
  type: Type.Literal("req"),    // 帧类型标识
  id: NonEmptyString,           // 请求 ID（用于关联响应）
  method: NonEmptyString,       // 方法名（如 "agent"、"sessions.list"）
  params: Type.Optional(Type.Unknown()),  // 参数
}, { additionalProperties: false });

// 响应帧：服务器 → 客户端
export const ResponseFrameSchema = Type.Object({
  type: Type.Literal("res"),    // 帧类型标识
  id: NonEmptyString,           // 对应的请求 ID
  ok: Type.Boolean(),           // 是否成功
  payload: Type.Optional(Type.Unknown()),  // 成功时的返回数据
  error: Type.Optional(ErrorShapeSchema),  // 失败时的错误信息
}, { additionalProperties: false });

// 事件帧：服务器 → 客户端（无需请求触发）
export const EventFrameSchema = Type.Object({
  type: Type.Literal("event"),  // 帧类型标识
  event: NonEmptyString,        // 事件名称
  payload: Type.Optional(Type.Unknown()),  // 事件数据
  seq: Type.Optional(Type.Integer({ minimum: 0 })),  // 序列号
  stateVersion: Type.Optional(StateVersionSchema),    // 状态版本
}, { additionalProperties: false });
```

> **衍生解释**：帧（Frame）在通信协议中指的是一个完整的数据单元。WebSocket 本身有帧的概念（文本帧、二进制帧等），但 OpenClaw 在此之上又定义了应用层帧。这种分层设计在网络协议中很常见——TCP 有自己的帧，HTTP 在 TCP 之上有自己的消息格式，OpenClaw 又在 WebSocket 之上定义了自己的消息格式。

这三种帧构成了一个**混合通信模式**：

```
┌──────────┐                          ┌──────────┐
│          │ ── req(id=1) ──────────▶│          │
│          │◀── res(id=1) ──────────│          │
│  Client  │                          │  Server  │
│          │ ── req(id=2) ──────────▶│          │
│          │◀── event(agent) ───────│          │  ← 服务器主动推送
│          │◀── event(agent) ───────│          │  ← 服务器主动推送
│          │◀── res(id=2) ──────────│          │
│          │◀── event(presence) ────│          │  ← 服务器主动推送
└──────────┘                          └──────────┘
```

请求-响应关联

请求和响应通过 `id` 字段关联。客户端发送请求时指定一个唯一 ID，服务器在响应中回传相同的 ID。这使得客户端可以在同一个连接上**同时发起多个请求**，并正确地将响应与请求对应起来。

> **衍生解释**：这种模式在协议设计中称为**多路复用**（Multiplexing）。HTTP/2 也使用了类似的设计（Stream ID）来在一个 TCP 连接上并发处理多个请求。与之对比，HTTP/1.1 需要按顺序处理请求（Head-of-line Blocking），或者通过打开多个连接来实现并发。

联合帧类型与判别器

OpenClaw 将三种帧合并为一个联合类型（Union Type），并使用 `type` 字段作为判别器（Discriminator）：

```
// src/gateway/protocol/schema/frames.ts
export const GatewayFrameSchema = Type.Union(
  [RequestFrameSchema, ResponseFrameSchema, EventFrameSchema],
  { discriminator: "type" },
);
```

判别器的作用是让下游代码生成工具（如 quicktype、Swift codegen）能生成精确的类型，而不是生成一个"所有字段都可选"的大型结构体。这一点在第 4.2.3 节的 Swift 代码生成中会详细讨论。

错误格式

当请求失败时，响应帧的 `error` 字段包含标准化的错误信息：

```
// src/gateway/protocol/schema/frames.ts
export const ErrorShapeSchema = Type.Object({
  code: NonEmptyString,           // 错误码，如 "INVALID_REQUEST"
  message: NonEmptyString,        // 人类可读的错误描述
  details: Type.Optional(Type.Unknown()),   // 额外的错误详情
  retryable: Type.Optional(Type.Boolean()), // 是否可重试
  retryAfterMs: Type.Optional(Type.Integer({ minimum: 0 })), // 重试间隔
}, { additionalProperties: false });
```

错误码定义在 `src/gateway/protocol/schema/error-codes.ts` 中：

```
export const ErrorCodes = {
  NOT_LINKED: "NOT_LINKED",       // 通道未连接
  NOT_PAIRED: "NOT_PAIRED",       // 设备未配对
  AGENT_TIMEOUT: "AGENT_TIMEOUT", // Agent 运行超时
  INVALID_REQUEST: "INVALID_REQUEST", // 请求格式错误
  UNAVAILABLE: "UNAVAILABLE",     // 服务不可用
} as const;
```

注意 `retryable` 和 `retryAfterMs` 字段——这是一种**协议级别的退避指导**（Backoff Guidance）。服务器可以告诉客户端："这个错误是暂时性的，你可以在 N 毫秒后重试"。这比让客户端自行猜测重试策略更加精确。

12.1.3 幂等键（Idempotency Key）机制防止重复操作

分布式系统中一个经典问题是**重复投递**：网络抖动导致客户端不确定请求是否已被服务器处理，于是重发请求，结果操作被执行了两次。对于 AI 助手来说，这意味着可能发出两条相同的回复消息，非常不理想。

> **衍生解释**：幂等性（Idempotency）是指一个操作执行一次和执行多次效果相同的性质。例如，HTTP 的 GET 请求天然是幂等的（获取同一个资源多少次结果都一样），而 POST 请求通常不是幂等的（每次 POST 都可能创建一个新资源）。幂等键是一种让非幂等操作变得幂等的技术手段。

OpenClaw 的幂等键设计

在 OpenClaw 协议中，涉及**副作用**的操作（发送消息、发起 Agent 对话、创建投票等）都要求客户端提供一个 `idempotencyKey` 字段：

```
// src/gateway/protocol/schema/agent.ts

export const SendParamsSchema = Type.Object({
  to: NonEmptyString,              // 发送目标
  message: NonEmptyString,         // 消息内容
  mediaUrl: Type.Optional(Type.String()),
  // ...其他字段省略
  idempotencyKey: NonEmptyString,  // ← 幂等键（必填）
}, { additionalProperties: false });

export const AgentParamsSchema = Type.Object({
  message: NonEmptyString,         // 用户消息
  agentId: Type.Optional(NonEmptyString),
  // ...其他字段省略
  idempotencyKey: NonEmptyString,  // ← 幂等键（必填）
}, { additionalProperties: false });

export const PollParamsSchema = Type.Object({
  to: NonEmptyString,
  question: NonEmptyString,
  options: Type.Array(NonEmptyString, { minItems: 2, maxItems: 12 }),
  // ...其他字段省略
  idempotencyKey: NonEmptyString,  // ← 幂等键（必填）
}, { additionalProperties: false });
```

幂等键的工作流程如下：

```
时序 1：正常请求
Client ── req(method="send", idempotencyKey="abc123") ──▶ Server
       ◀── res(ok=true) ──────────────────────────────── Server
        (Server 记住 "abc123" 已处理)

时序 2：重复请求（因为 Client 没收到 res，或网络重连后重发）
Client ── req(method="send", idempotencyKey="abc123") ──▶ Server
       ◀── res(ok=true) ──────────────────────────────── Server
        (Server 发现 "abc123" 已处理，直接返回之前的结果，不重复执行)
```

为什么不用请求 ID 来去重？

你可能会问：请求帧已经有 `id` 字段了，为什么还需要单独的幂等键？

原因在于**请求 ID 的生命周期与幂等键不同**。请求 ID 是一次 WebSocket 会话内的标识符——当 WebSocket 断开重连后，客户端会生成新的请求 ID。但幂等键由客户端预先生成并持久化，即使重连后也能使用相同的幂等键来标识"这是同一个业务操作"。

这种设计在 Stripe 等支付 API 中非常常见——它们也要求在创建支付请求时提供 `Idempotency-Key` HTTP 头，以防止因网络问题导致重复扣款。

幂等键的生成策略

OpenClaw 对幂等键的格式没有强制要求，只需要是非空字符串。在实际使用中，客户端通常使用 UUID v4 或类似的随机唯一标识符。关键点在于：

1. **同一个业务操作**在重试时使用**相同的**幂等键
2. **不同的业务操作**使用**不同的**幂等键
3. 幂等键在**合理的时间窗口内**不应重复

本节小结

OpenClaw 的协议设计体现了几个核心原则：

1. **选择合适的传输层**：WebSocket 的全双工能力天然适合 AI 助手的实时交互需求，同时保留 HTTP 端点满足兼容性。
2. **在传输层之上构建应用协议**：三种帧类型（请求、响应、事件）构成了一个灵活的混合通信模式，既支持经典的请求-响应，也支持服务器主动推送。
3. **面向可靠性设计**：幂等键机制确保了网络不稳定时的操作安全性，错误格式中的重试指导让客户端能做出更智能的决策。

在下一节中，我们将看到这些帧类型是如何使用 TypeBox 这一 Schema 系统来定义的，以及 OpenClaw 如何从单一定义中同时获得 TypeScript 类型安全和运行时校验。